

Sri Lanka Institute of Information Technology



IT3101 – Data Warehouse and Business Intelligence
Year 3, Semester 1 – 2026

Walmart Retail Sales Data Warehouse and Business
Intelligence System

Group No: 06

Student ID	Student Name
IT24103131	Illangakoon I. W. M. T. D.
IT24100646	Kumarathunga U. M. G. W. C. S.
IT24102202	Narampanawa E. M. S.
IT24103286	Rathnayake P. R. H. A.

01.Selection and Business Scenario Identification

Dataset Overview

Business Domain: Retail / E-commerce (large-format retail merchandising)

Purpose of the System:

The source data represents extracts from Walmart's operational retail systems specifically, weekly sales records aggregated by store and department, combined with regional economic indicators and promotional markdown activity. In a real Walmart store, this data would originate from point-of-sale (POS) and inventory management systems that record transactions at the individual sale level; the dataset we use represents a weekly aggregation of that transactional data, as would typically be extracted for reporting and forecasting purposes. This is supplemented with US public holiday reference data, which in a real business would come from an external calendar/reference data service integrated into planning systems.

Business Problem Addressed:

Large retail chains need to understand how store-level sales are influenced by seasonality, promotional markdown events, and macroeconomic conditions (unemployment, fuel prices, consumer price index) in order to plan staffing, inventory allocation, and marketing spend, particularly around holiday periods when demand fluctuates sharply. This dataset supports analysis of exactly that problem: which stores, departments, and time periods are most affected by these factors, and how holidays specifically drive sales spikes.

Dataset Source:

- **Primary:** Kaggle - "Walmart Store Sales Forecast" (originally published as the Walmart Recruiting - Store Sales Forecasting competition), sourced from real historical Walmart sales data. **Link:** <https://www.kaggle.com/datasets/aslanahmedov/walmart-sales-forecast>
- **Secondary:** Nager.Date Public Holiday API, an independent, freely accessible public API.
Links:
<https://date.nager.at/api/v3/PublicHolidays/2010/US>
<https://date.nager.at/api/v3/PublicHolidays/2011/US>
<https://date.nager.at/api/v3/PublicHolidays/2012/US>

Dataset Format:

- **Primary source:** CSV (three files: train.csv, stores.csv, features.csv)
- **Secondary source:** JSON (retrieved via REST API)

Number of Records:

- **train.csv:** 421,570 rows - weekly sales by store and department, spanning 45 stores over 143 weeks (5 February 2010 – 26 October 2012)
- **stores.csv:** 45 rows - one per store

- **features.csv:** 8,190 rows - weekly regional/economic data by store
- **Holiday JSON (2010–2012 combined):** 48 holiday records

Attributes Description:

File	Key Attributes	Description
train.csv	• Store • Dept • Date • Weekly_Sales • IsHoliday	• Unique Walmart store identifier • Department identifier • Week/date of the sales record • Weekly sales amount • Indicates whether the week is associated with a holiday
stores.csv	• Store • Type • Size	• Unique store identifier • Store type • Store size
features.csv	• Store • Date • Temperature • Fuel_Price • CPI • Unemployment • Markdown1–5 • IsHoliday	• Store identifier • Week/date • Regional temperature • Regional fuel price • Consumer Price Index • Unemployment rate • Promotional markdown values • Holiday indicator
holidays.json	• Date • localName • name • countryCode • fixed • global • types	• Holiday date • Local holiday name • Holiday name • Country code • Indicates whether the holiday is fixed • Indicates whether it applies globally • Holiday classification

Table 1.1

Why This Dataset Is Suitable for a DW/BI Solution

This combination provides a strong foundation for developing the required dimensional data warehouse and BI solution:

- **Represents an operational system:** the dataset represents data extracted from operational retail systems and provides an operational view of store-level sales activity.
- **Sufficient volume:** over 420,000 sales records provide meaningful statistical depth for trend analysis and aggregation.
- **Multiple dimensional attributes:** store type/size, department, date (with holiday flags), and economic conditions (temperature, fuel price, unemployment, CPI) all provide genuinely different angles for slicing the data.
- **Supports fact and dimension design:** Weekly_Sales forms a clear measurable fact at the grain of store + department + week. Store information, date information, department, and holiday-related attributes provide multiple dimensions or dimensional attributes for analysing sales from different perspectives.
- **Realistic business scenario:** forecasting and understanding retail sales performance is a genuine, common business need, directly addressed by this data.
- **Multiple genuinely independent sources:** the Kaggle sales/store/economic data and the Nager.Date holiday API are published by entirely separate, unrelated organizations, joined via a real, exact-match key (Date) not one dataset reformatted into a second file type.
- **Realistic data quality issues:** the MarkDown columns in features.csv contain genuine missing values (only populated from November 2011 onward), giving authentic material for the missing-value handling and data cleansing work required in Task 5, rather than artificially introduced mess.

02.Data Source Identification and Preparation

This identifies and documents the data sources used for the Walmart Data Warehouse and Business Intelligence solution. The project integrates structured CSV files containing sales, store, economic and promotional information with JSON files containing public holiday information.

A SQL Server staging schema named stg was created within the WalmartDW database to store and prepare the source data before loading it into the final data warehouse.

Data Sources

1. train.csv – Historical Sales Data

Description:

The primary transactional source containing historical weekly sales for Walmart departments across stores.

Source Format: CSV

Approximate Records: 421,570

Available Attributes:

Attribute	Description
Store	Walmart store identifier
Dept	Department identifier
Date	Date associated with the weekly sales
Weekly_Sales	Total weekly sales
IsHoliday	Indicates whether the week is associated with a holiday

Table 2.1

Natural Key: Store + Dept + Date

This dataset provides the main sales measure, Weekly Sales, for the analytical model.

2. stores.csv – Store Information**Description:**

Contains descriptive information about Walmart stores and their characteristics.

Source Format: CSV

Approximate Records: 45

Available Attributes:

Attribute	Description
Store	Unique store identifier
Type	Store category such as A, B or C
Size	Physical size of the store

Table 2.2

Natural Key: Store

The source attributes are later standardized as StoreID and StoreType during preparation.

3. features.csv – Economic and Promotional Features**Description:**

Contains external, economic, environmental and promotional information associated with Walmart stores and dates.

Source Format: CSV

Approximate Records: 8,190

Available Attributes:

Attribute	Description
Store	Walmart store identifier
Date	Feature date
Temperature	Average temperature
Fuel_Price	Local fuel price
Markdown1–5	Promotional markdown values

CPI	Consumer Price Index
Unemployment	Unemployment rate
IsHoliday	Holiday indicator

Table 2.3

Natural Key: Store + Date

This source provides additional context for analysing factors associated with changes in sales.

4. US - 2010.json, US - 2011.json, US - 2012.json – Public Holiday Data

Description:

These JSON files contain United States public holiday information for 2010, 2011 and 2012. They provide additional holiday names and classifications beyond the IsHoliday indicators available in the sales and features datasets.

Source Format: JSON

Approximate Raw Records: 16 records per year, 48 records in total.

Available Attributes:

Attribute	Description
HolidayDate	Date of the holiday
LocalName	Local name of the holiday
Name	Official holiday name
CountryCode	Country identifier
Fixed	Indicates whether the holiday has a fixed date
Global	Indicates whether the holiday applies nationally
Types	Holiday classification

Table 2.4

The three JSON sources are combined and prepared as a single holiday dataset.

Relationships Between Sources

The datasets are integrated using common business identifiers.

Source Relationship	Integration Key	Relationship
Train → Stores	Store	Many-to-One
Train → Features	Store + Date	Many-to-One
Train → Holidays	Date = HolidayDate	Date-based
Features → Stores	Store	Many-to-One

Table 2.5

The Train dataset acts as the primary sales source. Store information provides store characteristics, Features provides economic and promotional context, and the Holiday data provides additional calendar information.

Therefore, the main integration keys are:

- Store + Date

For example, a sales record can be enriched with store type, store size, temperature, fuel price, markdown values, CPI, unemployment and holiday information.

Data Preparation Steps

The source data was prepared using SQL Server before being used in the data warehouse.

CSV Preparation

The CSV files were loaded using BULK INSERT. The preparation included:

- Skipping CSV header rows.
- Using UTF-8 encoding.
- Checking missing values.
- Checking duplicate records.
- Validating Store and Department identifiers.
- Checking date ranges.
- Standardizing the IsHoliday field.
- Converting TRUE/FALSE holiday values into 1/0.
- Renaming attributes where required for warehouse readability.

Features Data Preparation

The features.csv file contains values such as NA in some numeric columns. Therefore, it was first loaded into:

stg.Features_Raw

The raw values were then converted into appropriate data types using TRY_CAST() and NULLIF().

Missing markdown values were converted to 0, while missing CPI and unemployment values were retained as NULL.

The prepared data was stored in:

stg.Features_Clean

Holiday Data Preparation

The JSON files were loaded using OPENROWSET and parsed using OPENJSON.

The three yearly files were combined into one dataset. Multiple holiday records occurring on the same date were handled during preparation, and the final prepared data was stored in:

stg.Holidays_Clean

Validation

The following checks were performed:

- Missing-value checking
- Duplicate detection
- Store identifier validation
- Store-Date relationship validation
- Date-range validation

- Holiday-date validation
- Referential-integrity checking
- Negative sales value checking

Negative Weekly_Sales values were retained because they may represent legitimate returns, reversals or sales adjustments.

Prepared Source Tables

The prepared data is maintained in the SQL Server staging area using the following tables:

Source	Prepared Table
train.csv	stg.Train_Clean
stores.csv	stg.Stores_Clean
features.csv	stg.Features_Clean
US – 2010/2011/2012.json	stg.Holidays_Clean

Table 2.6

An intermediate table, stg.Features_Raw, is also used to safely ingest the semi-cleaned CSV feature data before type conversion.

03.Data Warehouse Architecture Design

Purpose

This document defines the end-to-end architecture used to move Walmart sales data and public holiday data from their raw source formats into a structured data warehouse, and ultimately into Power BI dashboards and business insight reports. It shows how the outputs of every team member's task connect into a single, coherent analytical pipeline rather than four separate mini-projects.

System Architecture

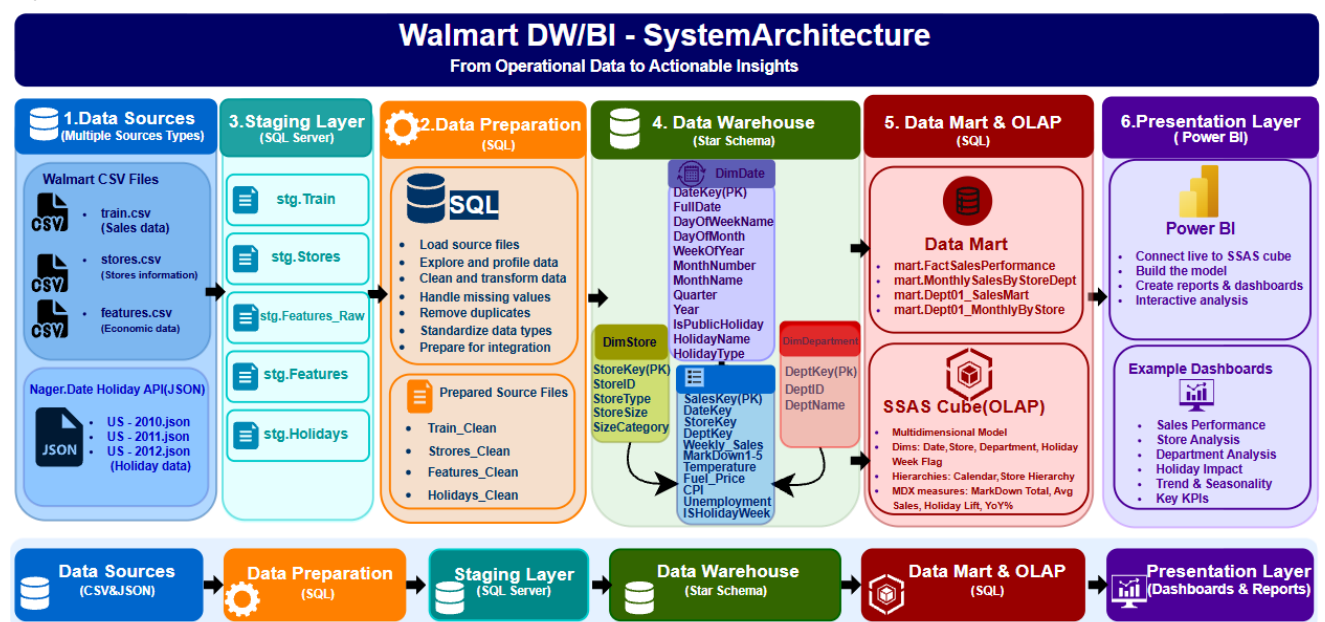


Figure 3.1

Layer-by-Layer Explanation

Data Sources (Multiple Source Types)

This is the raw, operational layer data as it exists before any DW/BI process touches it. Two source types feed the system:

- Walmart CSV files: train.csv (historical weekly sales by store/department), stores.csv (store metadata type and size), and features.csv (external/economic drivers temperature, fuel price, CPI, unemployment, and promotional markdowns).
- Nager.Date Holiday API (JSON): three yearly files (US-2010.json, US-2011.json, US-2012.json) providing US public holiday data used to enrich the sales data with holiday context.

Mixing CSV and JSON here demonstrates working with heterogeneous source formats, which is a deliberate design choice to show the pipeline can normalize different structures into one warehouse.

Staging Layer (SQL Server)

A holding area inside SQL Server where source data lands before transformation into the final dimensional model. It mirrors the source structure closely:

- stg.Train, stg.Stores, stg.Features_Raw, stg.Features, stg.Holidays

The staging layer's purpose is isolation raw/loading errors stay contained here and don't touch the production warehouse tables. It also allows re-running loads without re-touching the final model, and gives you an audit trail between "what came in" and "what got cleaned."

Data Preparation (SQL)

- Loads the raw source files into SQL Server
- Explores/profiles the data to understand its shape and quality
- Cleans and transforms values (e.g., converting text 'TRUE'/'FALSE' to 1/0 for IsHoliday)
- Handles missing values (e.g., the literal 'NA' text in Markdown columns converted to real NULLs, or imputed to 0)
- Removes duplicates (e.g., the Holidays deduplication by date)
- Standardizes data types (e.g., casting VARCHAR to DECIMAL/DATE)
- Produces prepared/cleaned output files: Train_Clean, Stores_Clean, Features_Clean, Holidays_Clean

This phase exists because raw source data is rarely analysis-ready it has to be validated and standardized before it can safely populate a warehouse.

Data Warehouse (Dimensional Model – Star Schema)

This is the analytical core of the system a star schema optimized for fast aggregation and reporting rather than transactional updates. It consists of:

- FactSales (the central fact table): the measurable business event (a store-department-week sales record), holding foreign keys to each dimension plus numeric measures Weekly_Sales, Markdown1-5, Temperature, Fuel_Price, CPI, Unemployment, IsHolidayWeek
- DimDate: calendar dimension with DateKey, FullDate, and date-hierarchy attributes (Day/Month/Quarter/Year) plus holiday flags enables time-based roll-up/drill-down.
- DimStore: store dimension with StoreKey, StoreID, StoreType, StoreSize, SizeCategory enables analysis by store characteristics.

- DimDepartment: department dimension with DeptKey, DeptID, DeptName enables analysis by department.

The star schema shape (fact table in the middle, dimensions radiating outward) is what makes OLAP operations (roll-up, drill-down, slice, dice, pivot) fast and intuitive later each dimension provides a natural hierarchy or filter axis to query along.

Data Mart & OLAP Layer

This layer provides analytical structures based on the Data Warehouse.

The Data Marts include:

- mart.FactSalesPerformance
- mart.MonthlySalesByStoreDept
- mart.Dept01_SalesMart
- mart.Dept01_MonthlyByStore

The **SSAS Multidimensional Cube** provides OLAP capabilities such as dimensions, hierarchies, measures, drill-down, and roll-up analysis.

Presentation Layer (OLAP & Power BI)

The consumption layer where business users interact with the warehouse:

- Power BI connects directly to the data warehouse, builds a semantic data model on top of the star schema, and lets analysts create reports, dashboards, and interactive visuals.
- Example dashboards listed represent the actual business questions the whole pipeline was built to answer: Sales Performance, Store Analysis, Department Analysis, Holiday Impact, Trend & Seasonality, and Key KPIs.

This is also the phase where the OLAP operations (roll-up, drill-down, slice, dice, pivot) that we discussed earlier actually get performed an analyst rolls up sales by month, drills down into a specific store, slices by holiday weeks, dices by store type and date range, or pivots store vs. department in a Power BI matrix visual.

04. Dimensional Data Warehouse Design and Implementation

A dimensional data warehouse was designed and implemented using **Microsoft SQL Server** to support Walmart weekly sales analysis. The warehouse uses a **Star Schema**, with a central fact table and three supporting dimension tables.

The dimensional model provides an analytical structure for examining sales by **date, store, store type, store size, department, holiday status, promotional markdowns, and economic factors**.

Schema Diagram

The implemented Star Schema consists of:

- Fact Table: dw.FactSales
- Dimension Tables: dw.DimDate, dw.DimStore, dw.DimDepartment
- The relationships are **one-to-many** from each dimension to FactSales.



Figure 4.1

Business Process:

Weekly retail sales performance tracking how much each department in each store sold per week, and what conditions (holidays, markdowns, economic climate) coincided with that week.

Grain:

Each row in FactSales represents the total sales of one department, in one store, for one specific week. This matches the grain of the cleaned source data (stg.Train_Clean) exactly, so no aggregation or disaggregation was needed when loading the fact table.

Measures:

Weekly_Sales and five Markdown columns (additive). Temperature, Fuel_Price, CPI, and Unemployment are also stored on the fact table but are non-additive context measures, since they describe regional conditions rather than sales activity.

Foreign Keys:

FactSales references three dimensions DimDate:DateKey, DimStore:StoreKey, DimDepartment:DeptKey each enforced with a REFERENCES constraint. All three joins are lossless: every (Store, Date) and (Store, Dept, Date) combination in the source data has a matching dimension row, so the fact load uses INNER JOIN throughout with no rows dropped.

Table Descriptions:

DimDate:

One row per calendar date (143 rows), with standard date-hierarchy attributes (day, week, month, quarter, year) plus holiday context. Holiday attributes (IsPublicHoliday, HolidayName, HolidayType) are denormalized directly onto DimDate rather than split into a separate DimHoliday dimension, since a date maps to at most one civic holiday a snowflaked table would add a join with no analytical benefit.

Attribute	Purpose
DateKey	Date key in YYYYMMDD format
FullDate	Full calendar date
DayOfWeekName	Day of the week name for the provided date
DayOfMonth	Day of the month
WeekOfYear	Week number
MonthNumber	Numeric month
MonthName	Month name
Quarter	Quarter number
Year	Calendar year
IsPublicHoliday	Indicates an exact public holiday
HolidayName	Public holiday name
HolidayType	Holiday or seasonal event associated with a transaction date

Table 4.1

DimStore:

One row per store (45 rows), with StoreType (A/B/C) and a derived SizeCategory (Small/Medium/Large, split at 50,000 and 150,000 sq ft) as a usable hierarchy above the raw StoreSize value.

Attribute	Purpose
StoreKey	Warehouse surrogate key
StoreID	Original source Store ID
StoreType	Store classification
StoreSize	Physical store size
SizeCategory	derived Small/Medium/Large classification based on StoreSize thresholds

Table 4.2

DimDepartment:

One row per department code (81 rows). The source data supplies only numeric department codes with no real names, so DeptName is a generated placeholder label rather than a genuine business name.

Attribute	Purpose
DeptKey	Warehouse surrogate key
DeptID	Original source department identifier
DeptName	Department name

Table 4.3

Schema Type: Star schema all three dimensions join directly to FactSales with no intermediate snowflaked tables.

FactSales:

dw.FactSales is the central fact table and stores the measurable sales records.

Attribute	Purpose
SalesKey	Surrogate primary key
DateKey	Foreign key to DimDate
StoreKey	Foreign key to DimStore
DeptKey	Foreign key to DimDepartment
Weekly_Sales	Weekly sales amount
MarkDown1–MarkDown5	Promotional markdown values
Temperature	Store/date temperature
Fuel_Price	Fuel price
CPI	Consumer Price Index
Unemployment	Unemployment rate
IsHolidayWeek	Walmart/Kaggle holiday-week indicator

Table 4.4

Design Assumptions:

The following assumptions were applied during dimensional modelling:

1. Fact Grain: Each fact record represents one department in one store for one weekly sales date.
2. Star Schema: A Star Schema was selected because it provides a simple structure with direct relationships between the fact and dimension tables and is suitable for BI and OLAP analysis.
3. Department Instead of Product: The source contains department information but no individual product information. Therefore, DimDepartment is used instead of creating an artificial Product dimension.
4. No Customer Dimension: The available sources do not contain customer-level information, so a Customer dimension was not created.
5. Store as Location Dimension: The source contains Store IDs and store characteristics but no geographical attributes. Therefore, DimStore represents the available location-related information.
6. Surrogate Keys: Surrogate keys are used for the Store and Department dimensions while the original source identifiers are retained as business keys.
7. Missing Markdown Values: Missing markdown values are treated as no recorded markdown activity and converted to zero during data preparation.
8. Missing Economic Values: Missing CPI and unemployment values are retained as NULL rather than being treated as zero.

9. **Negative Sales:** Negative weekly sales values are retained because they may represent returns, reversals, corrections or adjustments.
10. **Holiday Definitions:** Public holidays and Walmart/Kaggle holiday weeks are treated as separate concepts.
11. **Store Size Categories:** Store size may be grouped into Small, Medium and Large categories for analytical purposes based on the defined modelling thresholds.
12. **No Artificial Attributes:** Customer, product and geographical attributes are not artificially created because they are not available in the source data.

05.ETL Process Development

A complete **Extract, Transform and Load (ETL)** pipeline was developed using **Microsoft SQL Server** to integrate the Walmart sales, store, feature and holiday data into the dimensional data warehouse.

The ETL process consists of three main stages:

1. **Extraction** – Load data from CSV and JSON source files into staging tables.
2. **Transformation** – Clean, standardize and prepare the extracted data.
3. **Loading** – Load the transformed data into the dimension and fact tables.

Source Connection and Extraction Method

The source files are stored in a local dataset folder. The ETL pipeline receives the folder location through the parameter:

@FolderPath

This allows the same ETL process to be executed with a different source folder without changing the extraction logic.

The pipeline extracts six source files:

Source	Format	Staging Table
train.csv	CSV	stg.Train
stores.csv	CSV	stg.Stores
features.csv	CSV	stg.Features_Raw
US - 2010.json	JSON	stg.Holidays
US - 2011.json	JSON	stg.Holidays
US - 2012.json	JSON	stg.Holidays

Table 5.1

Extraction

Extraction method:

BULK INSERT for the three CSV sources (train.csv, stores.csv, features.csv) and OPENROWSET/OPENJSON for the three yearly holiday JSON files.

Source connection:

All sources are flat files read directly from disk by the SQL Server instance there is no live database or API connection involved, so "source connection" here is a file path (parameterised via @FolderPath) rather than a connection string.

Data extraction process:

Each source lands in a staging table with little or no transformation applied features.csv goes into an all-text Features_Raw table first, since its missing values are stored as the literal text "NA", which cannot be loaded directly into numeric columns. The extraction stage also converts this raw text into the final typed stg.Features table at this point, using TRY_CAST and NULLIF to turn "NA" into a real NULL, this type-casting step happens during extraction rather than transformation, since BULK INSERT cannot load "NA" text directly into numeric columns at all. The extraction procedure is fully re-runnable: staging tables are cleared with TRUNCATE at the start of every run, and the whole step executes inside a transaction, so a failure partway through cannot leave the staging area half-populated.

Transformation

The transformation stage is implemented through:

- stg.usp_Transform

The main cleaned staging tables are:

- stg.Train_Clean
- stg.Stores_Clean
- stg.Features_Clean
- stg.Holidays_Clean

Main transformations

Transformation	Implementation
"NA" handling (<i>performed during Extraction — see above</i>)	NULLIF()
Data type conversion (<i>performed during Extraction — see above</i>)	TRY_CAST()
Missing Markdown values	Converted to 0
Missing CPI/Unemployment	Retained as NULL
Holiday standardisation	Converted to 1/0
Whitespace removal	LTRIM() / RTRIM()
Case standardisation	UPPER()
Negative sales identification	Derived IsReturn
Holiday duplicate removal	ROW_NUMBER()

Store column standardisation	Column renaming
Monthly aggregation	SUM(), AVG(), COUNT()

Table 5.2

Sales Transformation

The IsHoliday field is standardized into:

- 1 = Holiday week
- 0 = Non-holiday week

An IsReturn attribute is also created:

- 1 when Weekly_Sales < 0
- 0 otherwise

Negative sales records are retained.

Features Transformation

"NA" values are converted to NULL and numeric values are safely converted using TRY_CAST().

Missing Markdown values are converted to zero, while missing CPI and Unemployment values remain NULL.

Store Transformation

Store attributes are standardized into:

- StoreID
- StoreType
- StoreSize

Holiday Transformation

Duplicate holiday dates are handled using ROW_NUMBER(). One record is retained for each holiday date.

Aggregation

A stg.Store_Monthly_Summary table is also created by grouping sales by:

- **Store + Year + Month**

It contains:

- Total Monthly Sales
- Average Weekly Sales
- Record Count
- Return Record Count

Loading

The loading stage is implemented through:

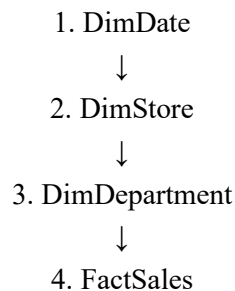
- dw.usp_Load

A **full refresh** strategy is used. Existing warehouse records are removed before the new data is loaded.

The dimensions are loaded before the fact table to maintain referential integrity.

Loading order:

Dimensions before the fact table, since FactSales's foreign keys depend on surrogate keys that only exist once the dimensions are populated enforced as:



Existing data is cleared first, in reverse (child-to-parent) order, since FactSales holds the foreign keys referencing the three dimensions.

Before the fact insert, the load explicitly checks for any Train_Clean row that wouldn't match a DimDate/DimStore/DimDepartment rows, a row silently dropped by an unmatched join is exactly the kind of failure that's easy to miss, so the load aborts loudly instead if that ever happens, rather than completing "successfully" with data quietly missing.

Validation Results

- Pipeline log:**

confirms all three steps ran in order and succeeded

	PipelineRunID	StepName	StartTime	EndTime	DurationSeconds	RowsAffected	Status
1	15E8A543-E3AB-4982-9B84-B743E1BAC932	EXTRACT	2026-09-30 00:32:01.7141780	2026-09-30 00:32:07.8936926	6	429853	SUCCESS
2	15E8A543-E3AB-4982-9B84-B743E1BAC932	TRANSFORM	2026-09-30 00:32:08.0070109	2026-09-30 00:32:11.0242655	3	431329	SUCCESS
3	15E8A543-E3AB-4982-9B84-B743E1BAC932	LOAD	2026-09-30 00:32:11.8663811	2026-09-30 00:32:30.5885918	19	421839	SUCCESS

Figure 5.1

- Row counts across the full chain:**

matched expectations exactly at every stage:

	TableName	Rows
1	stg.Train (raw)	421570
2	stg.Train_Clean	421570
3	dw.FactSales	421570
4	dw.DimDate	143
5	dw.DimStore	45
6	dw.DimDepartment	81
7	stg.Store_Monthly_Summary	1485

Figure 5.2

- Source-to-fact check:**

421,570 extracted vs. 421,570 loaded — MATCH, confirming no rows were silently lost across the full pipeline.

Results		Messages	
	RawExtracted	FinalLoaded	Result
1	421570	421570	MATCH

Figure 5.3

- **Derived Attribute Validation:**

1,285 rows flagged as returns, 420,285 not — matching the negative-sales count established in Task 2.

Results		Messages	
	IsReturn	RecordCount	
1	0	420285	
2	1	1285	

Figure 5.4

- **Aggregation integrity check:**

SUM(TotalMonthlySales) from Store_Monthly_Summary (6,737,218,987.11) exactly equals SUM(Weekly_Sales) from FactSales (6,737,218,987.11) — confirming the aggregation step neither lost nor double-counted any sales value.

100 %		No issues found		Ln: 407, Ch: 76 (160 chars, 2 lines)		SPC	LF
Results		Messages					
	Store	SalesYear	SalesMonth	TotalMonthlySales	AvgWeeklySales	RecordCount	ReturnRecordCount
1	1	2010	2	6307344.10	21749.462413	290	1
2	1	2010	3	5871293.98	20245.841310	290	1
3	1	2010	4	7422801.92	20561.778171	361	0
4	1	2010	5	5929938.64	20880.065633	284	0
5	1	2010	6	6084081.46	21728.862357	280	0
	GrandTotal						
1	6737218987.11						

Figure 5.5

- **Grain check:**

zero rows returned for duplicate (Store, Dept, Week) combinations in FactSales, confirming the fact table's grain held after the full reload.

Results		Messages	
DateKey	StoreKey	DeptKey	RecordCount

Figure 5.6

06.Data Mart Development

Two Data Marts were developed from the Walmart Enterprise Data Warehouse to provide simplified and focused analytical datasets:

1. **Chain-Wide Sales Performance Data Mart** – covers all stores and departments.
2. **Department 1 Sales Data Mart** – focuses specifically on Department 1.

The Data Marts are stored under the SQL Server mart schema and are derived from dw.FactSales, dw.DimDate, dw.DimStore, and dw.DimDepartment.

Purpose of the Data Marts

Chain-Wide Sales Performance Data Mart

The purpose is to provide a simplified, report-ready view of Walmart sales performance across all stores and departments. It combines sales, date, store, department, holiday, Markdown, and external-factor information into an analytical structure suitable for BI reporting.

Department 1 Sales Data Mart

The purpose is to provide a focused analytical environment containing only Department 1 sales information. It allows Department 1 performance to be analysed across stores, time periods, holidays, promotions, returns, and external factors without requiring users to work with the complete dataset.

	Detail table (weekly grain)	Summary table (monthly grain)
Part A — Chain-wide	mart.FactSalesPerformance	mart.MonthlySalesByStoreDept
Part B — Department 1 only	mart.Dept01_SalesMart	mart.Dept01_MonthlyByStore

Table 6.1

mart.FactSalesPerformance

Denormalizes the star schema into one table: every dimension attribute a report would filter or group by StoreType, SizeCategory, DeptName, MonthName, Quarter, HolidayName is already joined in, alongside all fact measures (Weekly_Sales, MarkDown1–5, Temperature, Fuel_Price, CPI, Unemployment). Two attributes are derived rather than copied: **IsReturn** (Weekly_Sales < 0) and **MarkdownTotal** (the sum of all five markdown columns), computed once here so downstream reports never recalculate them inconsistently. The table is indexed on StoreID, DeptID, and SaleDate, since those are the columns this audience actually filters and groups by.

mart.MonthlySalesByStoreDept

Aggregates that detail table to store/department/month, carrying forward TotalSales, AvgWeeklySales, WeekCount, HolidayWeekSales, TotalMarkdownSpend, and ReturnWeekCount.

mart.Dept01_SalesMart and mart.Dept01_MonthlyByStore

Mirror this exact structure, filtered to DeptID = 1, built *from Part A* rather than from the EDW directly, so there is no risk of the two marts drifting out of sync with each other.

Target Users

Chain-Wide Data Mart

- Senior management
- Regional and store managers
- Sales analysts

- Marketing and promotion teams
- BI developers and data analysts

Department 1 Data Mart

- Department 1 managers
- Store managers
- Merchandise/category managers
- Sales analysts
- Marketing and promotion teams
- BI users

Analytical Benefits

The Data Mart layer provides the following benefits:

- **Simplified reporting:** Important analytical attributes are available in report-ready tables, reducing the need for repeated joins.
- **Store performance analysis:** Stores can be compared by sales, store type, store size, and size category.
- **Department analysis:** Sales performance can be analysed across departments in the chain-wide Data Mart.
- **Department-focused analysis:** Department 1 users can analyse only the information relevant to their department.
- **Trend analysis:** Monthly and weekly sales trends can be analysed by store, department, and year.
- **Holiday analysis:** Holiday and non-holiday sales can be compared.
- **Promotion analysis:** Markdown activity can be examined alongside sales performance.
- **Return monitoring:** Negative sales records can be identified using the IsReturn indicator.
- **External-factor analysis:** Sales can be analysed together with temperature, fuel price, CPI, and unemployment.
- **Improved query performance:** Pre-aggregated monthly tables provide faster access to commonly required monthly analysis.

6. Validation Results

Run against the actual database, confirming both marts are correct and internally consistent:

Check	Result
mart.FactSalesPerformance row count	421,570 (matches dw.FactSales grain exactly)
mart.MonthlySalesByStoreDept row count	100,181 (of a theoretical max 120,285 confirms not every department is active in every store every month)
EDW total vs. flattened mart total	6,737,218,987.11 = 6,737,218,987.11 - MATCH
Flattened mart total vs. monthly aggregation total	6,737,218,987.11 = 6,737,218,987.11 - MATCH
mart.Dept01_SalesMart row count	6,435 rows, DistinctDepts = 1
Stores / weeks covered by Department 1	45 of 45 stores, 143 of 143 weeks
Dept01 mart total vs. filtered slice of Part A	MATCH
Dept01 monthly rollup vs. Dept01 mart total	MATCH

Table 6.2

Results

Messages

	TableName	RowCount
1	mart.FactSalesPerformance	421570
2	mart.MonthlySalesByStoreDept	100181

	EDW_Total	Mart_Total	Result
1	6737218987.11	6737218987.11	MATCH

	FlatTotal	AggregatedTotal	Result
1	6737218987.11	6737218987.11	MATCH

Figure 6.1

	RowCount	DistinctDepts	DeptID
1	6435	1	1

Figure 6.2

Results Messages			
	StoresCarryingThisDept	WeeksCovered	
1	45	143	

Figure 6.3

Results Messages			
	SourceSliceTotal	DeptMartTotal	Result
1	123638776.54	123638776.54	MATCH
	DeptMartTotal	MonthlyRollupTotal	(No column name)
1	123638776.54	123638776.54	MATCH

Figure 6.4

7. Markdown Effectiveness - A Corrected Finding

The markdown-effectiveness query is deliberately restricted to **SaleDate** >= '2011-11-01', since markdown data does not exist before that date (established in Task 2); including earlier weeks would conflate “markdown tracking did not exist yet” with “no markdown applied,” producing a misleading comparison. With that correction applied, average weekly sales *during* markdown weeks are actually slightly **lower** than non-markdown weeks across all three store size categories, the opposite of what an unrestricted comparison suggests:

Store Size	During Markdown	No Markdown	Difference
Small	8,146.84	8,416.69	-270
Medium	13,149.63	13,087.54	+62 (negligible)
Large	21,313.89	21,851.11	-537

Table 6.3

Results		Messages	
	SizeCategory	AvgSalesDuringMarkdown	AvgSalesNoMarkdown
1	Large	21313.886750	21851.105646
2	Medium	13149.630433	13087.535615
3	Small	8146.844384	8416.686097

Figure 6.5

This is correlational, not causal; a plausible explanation is that markdowns are applied during weeks already expected to be slower, rather than markdowns depressing sales but it demonstrates why controlling for the markdown program's actual start date matters before drawing conclusions from this data.

07. OLAP Analysis and Business Intelligence Dashboard Development

An interactive Business Intelligence dashboard was developed using Microsoft Power BI for the Walmart Data Warehouse solution.

The dashboard was designed to transform the dimensional data warehouse into an interactive analytical environment where users can monitor business performance, identify sales trends, compare store performance, and explore data using OLAP operations such as:

- Filtering
- Slicing
- Drill-down
- Roll-up
- Cross-filtering
- Hierarchical analysis

The dashboard consists of three main report pages:

1. Executive Summary
2. Trend Analysis
3. Interactive Analysis

The report uses the dimensional model developed in the previous tasks, particularly:

- Fact Sales
- Date Dimension
- Store Dimension
- Department Dimension
- Calculated Measures

Purpose of the Power BI Dashboard

The purpose of the dashboard is to provide business users with an easy-to-understand visual interface for analysing Walmart sales performance.

Instead of querying the Data Warehouse using SQL, managers and analysts can interact with charts, KPIs, cards, slicers and hierarchies.

The dashboard supports analysis of:

- Overall sales performance
- Average weekly sales
- Previous-year sales
- Year-over-year sales growth
- Holiday-week sales
- Store performance
- Department performance
- Promotional Markdown activity
- Sales trends over time
- Store type comparisons
- Department-level analysis

Target Users

The dashboard is designed for several categories of users.

Senior Management:

Senior managers can use the Executive Summary page to quickly monitor overall sales performance and major KPIs.

Store and Regional Managers:

Managers can compare sales between:

- Store Types
- Individual Stores
- Departments

Sales Analysts:

Sales analysts can use the Trend Analysis and Interactive Analysis pages to investigate changes in sales over time.

Marketing and Promotion Teams:

Marketing teams can compare Markdown promotional activity with sales performance.

Department Managers:

Department managers can filter the dashboard according to specific departments and evaluate performance across stores and time periods.

Report Page 1 – Executive Summary

Purpose

The Executive Summary page provides a high-level view of Walmart business performance.

The objective of this page is to allow managers to understand the current overall situation without having to examine detailed records.

It focuses on:

- Key Performance Indicators
- Summary Cards
- Overall Performance Indicators
- Year selection
- Year-over-Year comparison

Note: 2012 data in this dataset covers January to October only (as established in Task 1), so year-over-year comparisons involving 2012 reflect a partial year rather than a full 12 months.

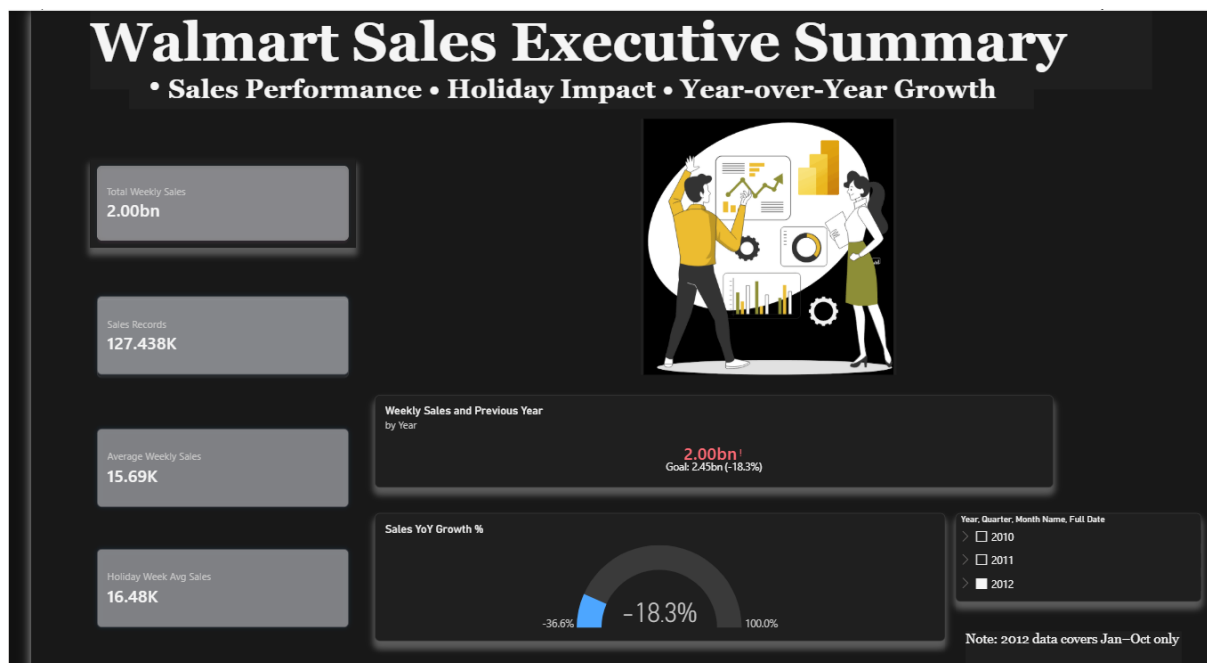


Figure 7.1

Report Page 2 – Trend Analysis

Purpose

The Trend Analysis page focuses on changes in sales performance over time and comparisons between different business measures.

This page satisfies the requirement for:

- Time-based analysis
- Line charts

- Comparison charts

The report uses Date and Store hierarchies to allow users to analyse performance at different levels of detail.

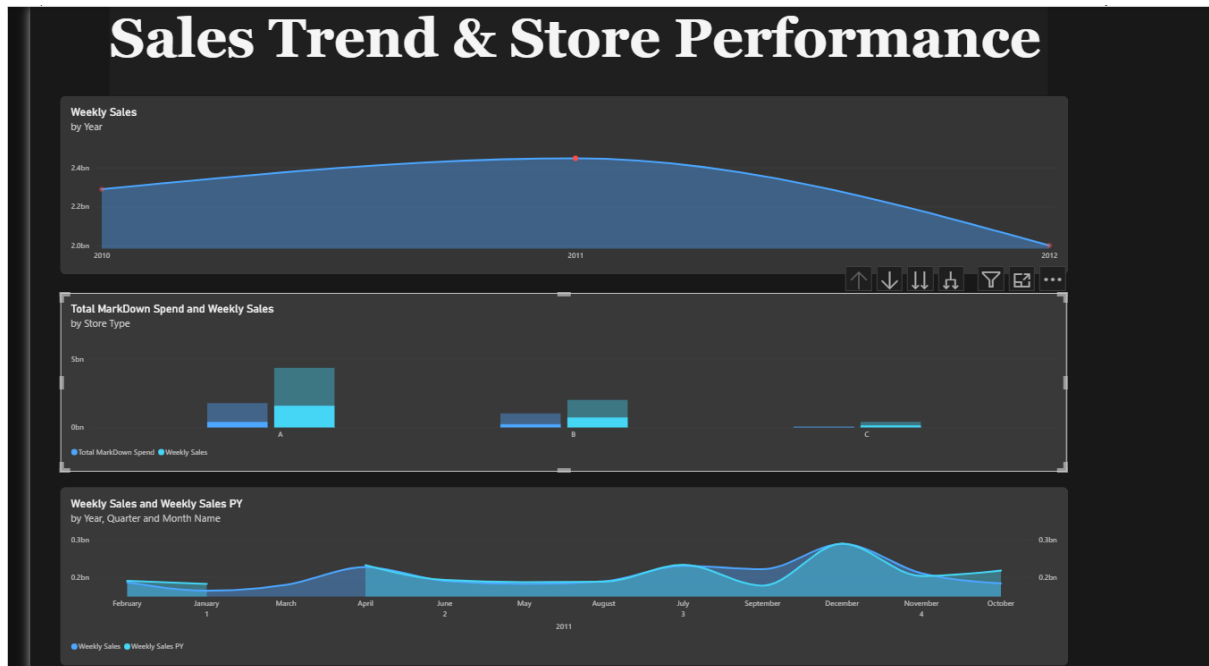


Figure 7.2

Report Page 3 – Interactive Analysis

Purpose

The Interactive Analysis page is designed for detailed exploratory analysis.

It contains:

- Slicers
- Filters
- Date hierarchy
- Store hierarchy
- Department selection
- Matrix analysis
- Drill-down
- Roll-up

This page allows the user to control the analytical context rather than viewing only predefined summaries.

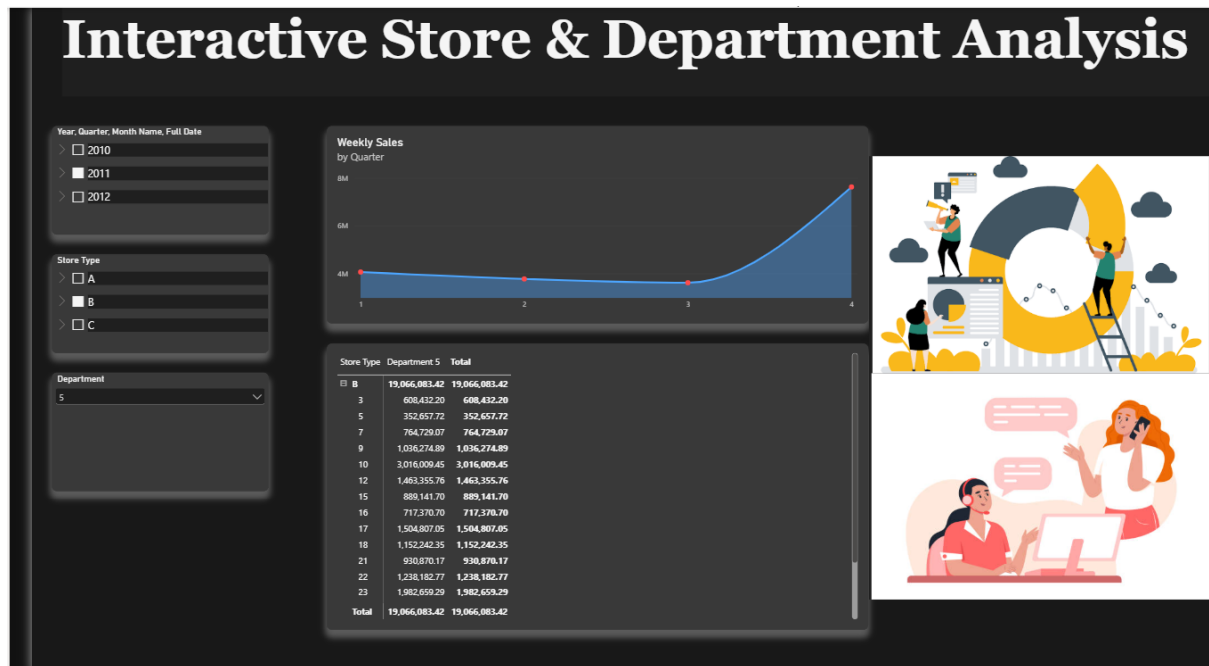


Figure 7.3

Dashboard: https://mysliit-my.sharepoint.com/:u:/g/personal/it24100646_my_sliit_lk/IQA-6Z7oaEOJT7X5Z3ZyEUxqAVPM-II2eOILsSTW_Nk6BaE?e=ibFHFU

08. Dashboard Analysis and Business Insights

The Power BI dashboard developed in Task 7 was analysed to identify meaningful business insights from the Walmart Data Warehouse.

The analysis focuses on the main areas represented in the dashboard:

- Overall sales performance
- Sales trends over time
- Holiday-week performance
- Store performance
- Department performance
- Markdown and promotional activity
- Operational planning

The purpose of this analysis is not only to describe dashboard visuals, but also to identify patterns that can support management decision-making.

The Walmart dataset contains approximately 421,570 weekly store-department sales records across 45 stores and 81 departments, covering approximately February 2010 to October 2012. This analysis of our own dashboard found that holiday weeks generate higher average sales, Type A stores generally outperform Type C stores, and November/December contain important seasonal sales peaks.

Insight 1 – Sales Show Clear Seasonal Patterns

Observation

- The Trend Analysis page shows that sales are not evenly distributed throughout the year.
- The Weekly Sales line chart contains noticeable increases during particular periods, especially toward the final months of the year.
- November and December are important sales periods because of major shopping events and holidays.
- Our dashboard's Trend Analysis page identifies November and December as peak sales months driven by holiday shopping
- This indicates that Walmart experiences significant seasonality.

Business Meaning

Demand increases considerably during certain periods rather than remaining constant throughout the year. This means that normal inventory and staffing levels may not be sufficient during peak periods. If Walmart prepares only according to average weekly demand, stores may experience:

- Product shortages
- Stock-outs
- Longer checkout times
- Increased pressure on employees
- Lost sales opportunities
- Business Decision Support

Management can use this insight for seasonal demand planning. Before historically high-performing periods, Walmart can:

- Increase inventory levels.
- Increase staffing levels.
- Improve warehouse replenishment.
- Prepare additional checkout capacity.
- Schedule promotional campaigns before the expected demand increase.

Historical sales trends should therefore be incorporated into future sales forecasting and operational planning.

Insight 2 – Holiday Weeks Generate Stronger Sales Performance

Observation

- The Executive Summary includes a:
 - Holiday Week Avg Sales KPI.
- The dashboard also contains the IsHolidayWeek attribute, allowing holiday and non-holiday weeks to be compared.
- Our dashboard shows that average sales during holiday weeks are approximately 7% higher than ordinary weeks.
- This suggests that important holiday periods create additional customer demand.

Business Meaning

Holiday periods are strategically important sales opportunities. The increase can be associated with:

- Increased customer spending
- Seasonal product demand
- Holiday-related shopping
- Special promotions
- Increased store traffic

However, stronger demand also increases operational pressure.

Business Decision Support

Walmart management should treat holiday weeks as priority periods. Possible actions include:

- Increasing stock before major holiday weeks.
- Scheduling additional employees.
- Improving supplier coordination.
- Increasing promotional activities.
- Ensuring high-demand departments have sufficient inventory.
- Closely monitoring stock levels during holiday periods.

The dashboard's Holiday Week KPI can also be monitored from year to year to determine whether holiday performance is improving.

Insight 3 – Store Types Do Not Produce Equal Sales Performance

Observation

- The Trend Analysis page compares sales according to the Store hierarchy:

Store Type → Store ID

- The Store Type comparison indicates noticeable differences between:
 - Type A
 - Type B
 - Type C

Our Store Type comparison shows that Type A stores generally record considerably higher average weekly sales than Type C stores.

Store size also has a strong relationship with total sales in this dataset.

Business Meaning

The result suggests that store characteristics affect sales performance. Larger or higher-volume Store Types may naturally require:

- More inventory
- Greater staffing capacity
- Larger replenishment volumes
- Different promotion strategies

Therefore, applying the same operational strategy to every store may not be efficient.

Business Decision Support

Management should use store-specific planning instead of treating all stores equally. For example:

High-performing stores Management can:

- Allocate additional stock.
- Increase staffing during peak periods.
- Prioritize fast replenishment.
- Protect availability of high-demand products.
- Lower-performing stores
- Management can investigate:
 - Product mix
 - Local demand
 - Store size
 - Department performance
 - Promotional effectiveness

The Power BI Store hierarchy can then be used to drill from:

Store Type → Store ID

to determine whether weak performance is related to the whole Store Type or only particular stores.

Insight 4 – Store Size Is an Important Performance Factor

Observation

The Data Warehouse contains:

- Store Size
- Store Type
- Size Category

Store Size Categories were created as:

- Small
- Medium
- Large

This analysis of our own dashboard shows a strong association between store size and overall sales performance. Our analysis found a correlation of approximately 0.85 between store size and total sales.

This suggests that larger stores generally have greater sales capacity.

Business Meaning

Larger stores may generate higher sales because they can potentially provide:

- More departments
- Greater product variety
- Higher inventory capacity
- More customer capacity

However, larger stores may also have higher operating costs. Therefore, high total sales alone should not automatically be interpreted as greater efficiency.

Business Decision Support

Management can use this insight when comparing stores. Instead of evaluating every store only using total Sales. Additional measures should be considered, such as:

- Sales relative to Store Size

- Average Department Sales
- Sales growth
- Holiday performance

Insight 5 – Department Performance Varies Significantly

Observation

The Interactive Analysis page contains a Department slicer and a Matrix using:

- Rows: Store Type → Store ID
- Columns: Department
- Values: Weekly Sales

This analysis shows that sales are not evenly distributed across all departments. Some departments contribute significantly more sales than others, and the same department may perform differently across different stores.

Business Meaning

Individual departments have different levels of demand. A single company-wide inventory strategy may therefore lead to:

- Excess stock in low-demand departments
- Insufficient stock in high-demand departments
- Inefficient use of store space
- Reduced availability of popular products

Department performance can also change between stores because of differences in local demand.

Business Decision Support

Department managers should use department-level sales information when planning:

- Inventory
- Replenishment
- Store space
- Promotions
- Seasonal stock

The Department slicer can be combined with Store and Year filters.

For example: Department = Department 1, Store Type = A, Year = 2012

This enables managers to determine exactly how a particular department performs under a particular business context. High-performing departments can receive more inventory and shelf space, while low-performing departments can be investigated further.

Insight 6 – Promotional Markdown Activity Should Be Targeted Rather Than Applied Uniformly

Observation

The Trend Analysis page contains a comparison between:

- Weekly Sales
- Total Markdown Spend

The purpose of this comparison is to determine whether periods or stores with greater promotional Markdown activity also show improved sales. Markdown information does not remain constant across all stores and time periods. Therefore, promotions appear to be used selectively rather than continuously. The source Features dataset also shows that Markdown information is heavily concentrated in later periods of the dataset rather than being available across the entire historical period.

Business Meaning

Markdown spending can potentially support demand, but simply increasing discounts does not automatically guarantee higher sales. The effectiveness of promotions may depend on:

- Store
- Department
- Month
- Holiday period
- Product demand

Therefore, promotional performance should be evaluated rather than assuming that more Markdown expenditure is always beneficial.

Business Decision Support

Marketing managers should analyse Markdown activity together with sales before planning future promotions. They can use the dashboard to investigate questions such as:

- Which Store Types respond well to promotions?
- Which departments show sales increases during Markdown periods?
- Are Markdown campaigns more effective during holiday periods?
- Are some stores receiving promotions without meaningful sales improvement?

Future promotions can then be concentrated on Store-Department combinations where previous campaigns produced stronger results.

Insight 7 – Negative Sales Records Indicate Returns or Adjustments That Should Be Monitored

Observation

During data preparation, approximately: 1,285 records contained negative Weekly Sales values. These records were deliberately retained rather than deleted. They were classified using the derived attribute: IsReturn. This matches the 1,285 negative sales records identified during data preparation in Task 2.

Business Meaning

Negative sales may represent:

- Product returns
- Refunds
- Corrections
- Transaction reversals

Although these records represent only a small proportion of the complete dataset, repeated negative sales in a particular Store or Department may indicate an operational issue.

Business Decision Support

Management can monitor return activity according to:

- Store
- Department
- Month
- Store Type

If one Department or Store records unusually high return levels, managers can investigate:

- Product quality
- Incorrect pricing
- Customer dissatisfaction
- Incorrect transactions
- Operational errors

A future dashboard improvement could include a dedicated:Return Rate KPI to identify stores or departments requiring investigation.

Insight	Dashboard Evidence	Business Decision
Sales are seasonal	Time-based sales trend	Increase stock and staffing before peak periods
Holiday weeks perform strongly	Holiday Week Avg Sales	Prioritize inventory and operations for holidays
Store Types perform differently	Store comparison chart	Use store-specific strategies
Store size influences performance	Store attributes and comparison	Compare performance relative to store size
Departments have different demand	Department Matrix and slicer	Optimize inventory by department
Markdown effectiveness varies	Sales vs Markdown comparison	Target promotions more carefully
Negative sales require monitoring	IsReturn / negative sales records	Investigate return and adjustment patterns
YoY growth reveals performance changes	YoY Growth and PY KPIs (2012 partial year)	Detect and investigate decline earlier
Drill-down identifies root causes	Interactive Analysis page	Take targeted rather than company-wide action

Table 8.1

Overall Conclusion

This project successfully developed a complete **Walmart Retail Sales Data Warehouse and Business Intelligence solution**, covering the major stages of a Data Warehouse and BI implementation.

First, the Walmart retail dataset and supporting public holiday data were identified and analysed to define the business requirements and analytical objectives. The data sources, formats, attributes, relationships, and preparation requirements were documented. A suitable **Data Warehouse architecture** was then designed, following a layered approach from source data and preparation through staging, the Data Warehouse, OLAP, and BI reporting.

A **star-schema Data Warehouse** was implemented using FactSales, DimDate, DimStore, and DimDepartment. The design provides a structured environment for analysing sales by time, store, and

department. An ETL process was developed using SQL Server to extract the CSV and JSON sources, transform and clean the data, handle missing values and duplicates, standardise data, and load the dimensions and fact table. Validation and reconciliation procedures were used to ensure data consistency and integrity.

The project also developed **Data Marts** to support specific analytical requirements. The Chain-Wide Sales Performance Data Mart provides a simplified view of sales across all stores and departments, while the Department 1 Data Mart provides a focused environment for Department 1 analysis. These Data Marts support sales trends, store and department comparisons, holiday analysis, Markdown analysis, return monitoring, and analysis of external factors such as temperature, fuel price, CPI, and unemployment.

Finally, the solution provides a foundation for **OLAP and Business Intelligence analysis**, enabling the prepared warehouse and Data Mart information to be used for interactive reporting and dashboards. Overall, the project demonstrates how raw retail data can be transformed into a structured and integrated analytical environment that supports efficient reporting, meaningful analysis, and data-driven business decision-making.

Use of AI

AI tools were used as a **supporting tool for learning and development** throughout this assessment. They were used for brainstorming ideas, improving explanations, refining documentation, and assisting with debugging and understanding technical issues.

All AI-generated suggestions were **critically reviewed, verified, and adapted** based on the actual implementation of the project. The final architecture, SQL scripts, ETL processes, Data Warehouse, Data Marts, OLAP implementation, analysis, and results were developed and validated by the students.

AI tools were not used to replace the students' technical work, generate false results, or fully generate the assignment. The students remained responsible for understanding, implementing, testing, and verifying the submitted work. This use of AI follows the **CLEAR framework** and the assessment guidelines for responsible and transparent AI use.